Université Sultan Moulay Slimane

Faculté des Sciences et Techniques de Béni Mellal

Master Intelligence Artificielle et Digital Computing

TP 2 : Simulation Multi-Agent

Formation de l'opinion publique avec Mesa3

Étudiante : DOGHMI Nassima

Année universitaire : 2026–2027

Année universitaire 2025–2026

Table des matières

Introduction **3**

1 Présentation du modèle **4**
1.1 Phénomène étudié 4
1.2 Objectif du TP 4
1.3 Protocole ODD 4
1.3.1 Overview 4
1.3.2 Règle d'évolution de l'opinion 5
1.3.3 Concepts de conception 5

2 Environnement de développement **6**
2.1 Technologies utilisées 6
2.2 Importation des bibliothèques 6

3 Implémentation de l'agent **7**
3.1 Classe CitizenAgent 7
3.2 Initialisation de l'agent 7
3.3 Interaction avec les voisins 7

4 Implémentation du modèle **9**
4.1 Classe OpinionModel 9
4.2 Création de la grille 9
4.3 Création des agents 9
4.4 Collecte des données 10
4.4.1 Données du modèle 10
4.4.2 Données individuelles 11

5 Exécution de la simulation **12**
5.1 Paramètres 12
5.2 Collecte des résultats 13

6 Visualisation et analyse des résultats **14**
6.1 Évolution globale de l'opinion 14
6.1.1 Interprétation 14
6.2 Trajectoires des opinions individuelles 15

7 Analyse des questions **16**
7.1 Impact du paramètre α 16
7.2 Pourquoi la polarisation est-elle difficile dans ce modèle ? 17
7.3 Émergence 18
7.4 Activation des agents dans Mesa 3 18
7.5 Différence entre model_reporters et agent_reporters 18
7.6 Effet de `torus=True` 19

8 Ouverture et amélioration du modèle **20**
8.1 Introduction d'un biais de confirmation 20
8.2 Vers un véritable réseau social 20

Conclusion **21**

Introduction

Les systèmes multi-agents permettent de modéliser des phénomènes complexes à partir de l'interaction entre plusieurs entités autonomes appelées agents.

Dans ce travail pratique, nous utilisons le framework Python **Mesa 3** pour construire une simulation multi-agent portant sur la **formation de l'opinion publique**. Chaque agent représente un individu possédant une opinion qui peut évoluer en fonction des opinions de ses voisins.

L'objectif est d'étudier comment des interactions locales entre individus peuvent conduire à un comportement collectif, notamment à la convergence vers un consensus.

Le TP permet également de découvrir la nouvelle syntaxe de Mesa 3, notamment la gestion des agents avec `self.agents.shuffle_do("step")`, l'utilisation d'une grille `MultiGrid` et la collecte des données avec `DataCollector`.

Chapitre 1

Présentation du modèle

1.1 Phénomène étudié

Le phénomène étudié est la formation de l'opinion publique dans une population.

Dans une société, les individus interagissent avec leur entourage et peuvent modifier progressivement leur opinion sous l'influence des autres individus.

L'objectif de la simulation est donc d'étudier comment ces interactions locales peuvent produire un comportement collectif tel qu'un consensus, une polarisation ou plusieurs groupes d'opinion.

1.2 Objectif du TP

Les principaux objectifs de ce TP sont :

— découvrir la modélisation basée sur les agents avec Mesa 3 ;
— représenter une population d'individus par des agents ;
— modéliser l'évolution des opinions individuelles ;
— étudier l'influence des voisins sur chaque agent ;
— analyser l'évolution de l'opinion moyenne ;
— mesurer la dispersion des opinions ;
— observer la convergence du système ;
— comprendre la notion d'émergence dans un système multi-agent.

1.3 Protocole ODD

Le protocole ODD permet de décrire de manière structurée le modèle multi-agent.

1.3.1 Overview

Le modèle contient les éléments suivants :

— **Agents :** les agents représentent des individus ;
— **Opinion :** chaque agent possède une opinion comprise entre 0 et 1 ;
— **Grille :** les agents sont placés dans une grille spatiale ;
— **Influence :** chaque agent est influencé par les opinions de ses voisins ;
— **Temps :** la simulation évolue par pas de temps appelés *ticks*.

L'opinion d'un agent est définie par :

$$o_i \in [0, 1]$$

où :
— 0 représente une opinion totalement contre ;
— 0.5 représente une opinion neutre ;
— 1 représente une opinion totalement pour.

1.3.2 Règle d'évolution de l'opinion

À chaque étape, l'agent observe les opinions de ses voisins et calcule leur moyenne. La nouvelle opinion est calculée selon la formule :

$$o_i^{t+1} = o_i^t + \alpha \left(\overline{o}_{\mathcal{N}_i}^t - o_i^t \right)$$

où :
— o_i^t est l'opinion actuelle de l'agent ;
— o_i^{t+1} est sa nouvelle opinion ;
— $\overline{o}_{\mathcal{N}_i}^t$ est la moyenne des opinions de ses voisins ;
— α est le coefficient d'influence ;
— $\alpha \in [0, 1]$.

Ainsi, plus α est grand, plus l'agent se rapproche rapidement de l'opinion moyenne de ses voisins.

1.3.3 Concepts de conception

Le modèle met en évidence plusieurs concepts importants :
— **Émergence :** le comportement global apparaît à partir des interactions locales entre agents ;
— **Interaction :** chaque agent adapte son opinion en fonction de ses voisins ;
— **Stochasticité :** les opinions initiales sont générées aléatoirement ;
— **Adaptation :** les agents modifient progressivement leur opinion sous l'effet de l'influence sociale.

Chapitre 2

Environnement de développement

2.1 Technologies utilisées

Le TP est réalisé en Python avec le framework **Mesa 3**, spécialement conçu pour la modélisation et la simulation de systèmes multi-agents.

Les principales bibliothèques utilisées sont :

— **Mesa :** création et gestion des agents et du modèle ;
— **NumPy :** calcul numérique et calcul des moyennes et variances ;
— **Matplotlib :** génération des graphiques ;
— **Pandas :** manipulation des données collectées.

2.2 Importation des bibliothèques

Le code utilisé pour importer les bibliothèques est :

```
# Importation des biblioth ques n cessaires
import mesa
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Configuration de l'affichage des graphiques
%matplotlib inline
```

Ces bibliothèques permettent de construire le modèle, effectuer les calculs, collecter les résultats et visualiser l'évolution du système.

Chapitre 3

Implémentation de l'agent

3.1 Classe CitizenAgent

La classe `CitizenAgent` représente un individu de la population.
Chaque agent possède deux principales caractéristiques :

— son opinion ;

— son coefficient d'influence α.

3.2 Initialisation de l'agent

L'opinion initiale est générée aléatoirement entre 0 et 1.

```
class CitizenAgent(mesa.Agent):

    def __init__(self, model, alpha):
        super().__init__(model)

        self.opinion = self.model.random.random()
        self.alpha = alpha
```

L'utilisation de :

```
self.model.random.random()
```

permet d'attribuer à chaque agent une opinion initiale différente.

3.3 Interaction avec les voisins

À chaque étape, l'agent récupère ses voisins présents autour de lui dans la grille.

```
neighbors = self.model.grid.get_neighbors(
    self.pos,
```

```
    moore=True,
    include_center=False
)
```

Le voisinage de Moore considère les huit cases entourant l'agent.

L'agent calcule ensuite la moyenne des opinions de ses voisins :

```
mean_opinion = np.mean(
    [neighbor.opinion for neighbor in neighbors]
)
```

Enfin, son opinion est mise à jour avec :

```
self.opinion = self.opinion + \
    self.alpha * (mean_opinion - self.opinion)
```

Cette formule permet de rapprocher progressivement l'opinion de l'agent de celle de son voisinage.

Chapitre 4

Implémentation du modèle

4.1 Classe OpinionModel

La classe `OpinionModel` représente le système global dans lequel les agents évoluent. Le modèle crée une grille et y place les agents.

4.2 Création de la grille

La grille utilisée est une grille `MultiGrid` :

```
self.grid = mesa.space.MultiGrid(
    width,
    height,
    torus=True
)
```

Le paramètre :

```
torus=True
```

permet de considérer la grille comme un espace avec des bords communicants. Ainsi, les agents situés sur un bord peuvent également avoir des voisins de l'autre côté de la grille.

4.3 Création des agents

Un agent est créé pour chaque case de la grille :

```
for _, x, y in self.grid.coord_iter():

    agent = CitizenAgent(
        self,
```

```
            self.alpha
        )

        self.grid.place_agent(
            agent,
            (x, y)
        )
```

Pour une grille de dimension 10×10, le modèle contient donc :

$$10 \times 10 = 100$$

agents.

4.4 Collecte des données

Le modèle utilise `DataCollector` afin d'enregistrer les résultats de la simulation.
Deux types de données sont collectés :

— les données globales du modèle ;

— les données individuelles des agents.

4.4.1 Données du modèle

Les variables globales sont :

— l'opinion moyenne ;

— la variance des opinions.

Le code correspondant est :

```
model_reporters={
    "Opinion Moyenne":
        lambda m: np.mean(
            [a.opinion for a in m.agents]
        ),

    "Variance (Dispersion)":
        lambda m: np.var(
            [a.opinion for a in m.agents]
        )
}
```

L'opinion moyenne permet d'observer la tendance générale de la population.
La variance permet de mesurer la dispersion des opinions autour de leur moyenne.

4.4.2 Données individuelles

Pour chaque agent, son opinion est également enregistrée :

```
agent_reporters={
    "Opinion": "opinion"
}
```

Cette information permet ensuite d'observer les trajectoires individuelles des agents.

Chapitre 5

Exécution de la simulation

5.1 Paramètres

Les paramètres utilisés dans le TP sont :

TABLE 5.1 – Paramètres de la simulation

Paramètre	**Valeur**
Largeur de la grille	10
Hauteur de la grille	10
Nombre d'agents	100
Coefficient d'influence α	0.5
Nombre de ticks	50
Graine aléatoire	42

Le modèle est créé avec :

```
model = OpinionModel(
    width=WIDTH,
    height=HEIGHT,
    alpha=ALPHA,
    seed=42
)
```

La simulation est ensuite exécutée pendant 50 ticks :

```
for i in range(STEPS):
    model.step()
```

5.2 Collecte des résultats

Après l'exécution, les données sont récupérées sous forme de `DataFrame` :

```
model_data = \
    model.datacollector.get_model_vars_dataframe()

agent_data = \
    model.datacollector.get_agent_vars_dataframe()
```

Ces données permettent de réaliser l'analyse graphique de la simulation.

Chapitre 6

Visualisation et analyse des résultats

6.1 Évolution globale de l'opinion

Le premier graphique représente simultanément :

— l'opinion moyenne de la population ;

— la variance des opinions.

Le graphique permet donc d'étudier à la fois la tendance générale et la dispersion des opinions.

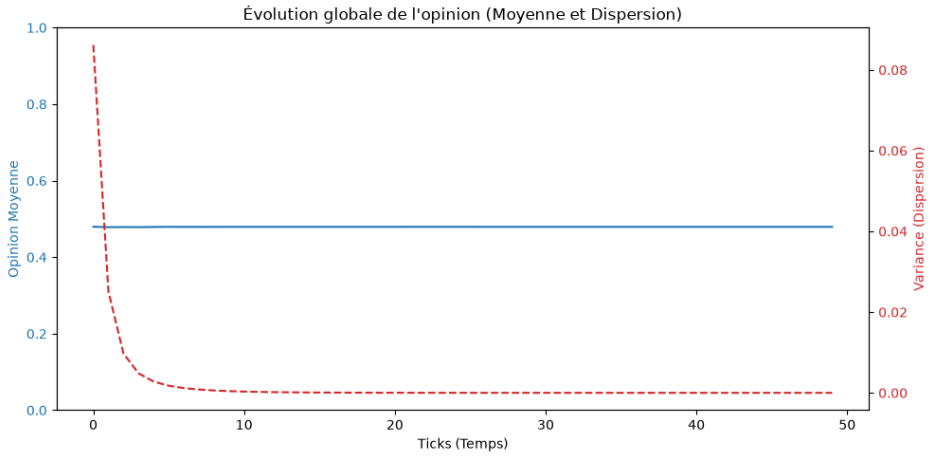


FIGURE 6.1 – Évolution globale de l'opinion moyenne et de la dispersion

6.1.1 Interprétation

Au début de la simulation, les opinions sont distribuées aléatoirement entre 0 et 1.

Au fur et à mesure des interactions, chaque agent rapproche son opinion de celle de ses voisins.

Avec $\alpha = 0.5$, l'agent prend en compte une partie importante de l'écart entre sa propre opinion et la moyenne de son voisinage.

La variance permet d'observer la diminution de la dispersion des opinions. Une diminution progressive de cette variance indique que les opinions deviennent de plus en plus proches.

Ainsi, la dynamique du modèle conduit progressivement vers une situation de consensus global.

6.2 Trajectoires des opinions individuelles

Le deuxième graphique représente l'évolution des opinions d'un échantillon de 20 agents.

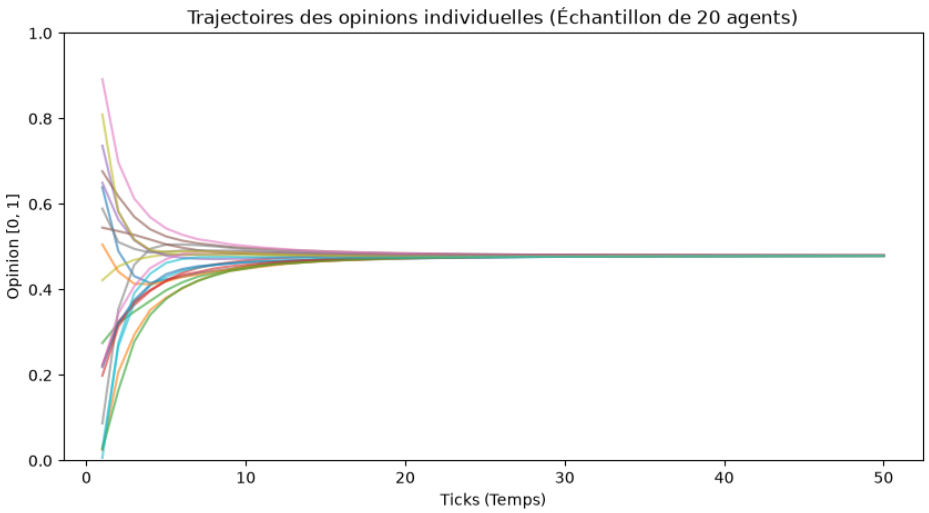


FIGURE 6.2 – Trajectoires des opinions individuelles

Chaque courbe représente l'évolution de l'opinion d'un agent au cours des ticks.

On peut observer que les différentes opinions évoluent progressivement vers des valeurs proches.

Cette convergence est une conséquence directe de la règle de mise à jour :

$$o_i^{t+1} = o_i^t + \alpha \left(\overline{o}_{\mathcal{N}_i}^t - o_i^t \right)$$

L'interaction répétée entre les agents provoque donc une réduction progressive des différences d'opinion.

Chapitre 7

Analyse des questions

7.1 Impact du paramètre α

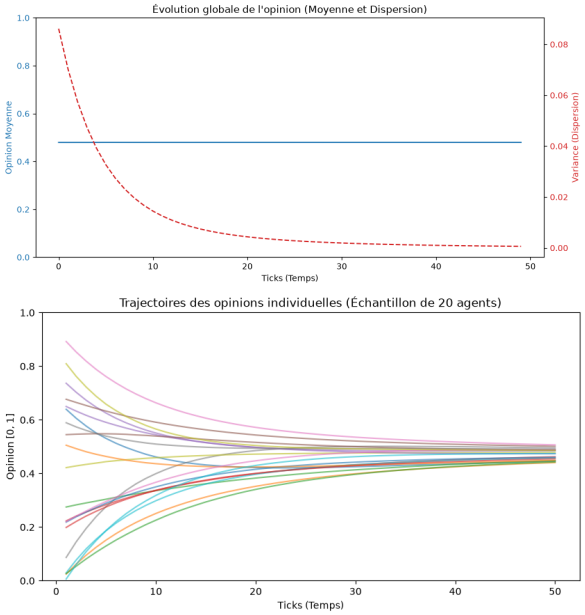


FIGURE 7.1 – $\alpha \approx 0$

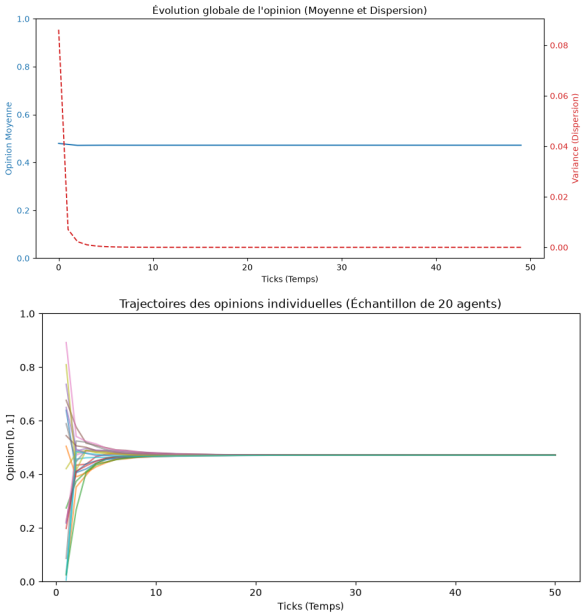


FIGURE 7.2 – $\alpha = 1$

Le paramètre α contrôle l'intensité de l'influence sociale.

Si α est proche de 0, l'agent modifie très peu son opinion à chaque étape. La convergence est donc lente.

À l'inverse, lorsque α est égal à 1, l'agent adopte directement l'opinion moyenne de ses voisins.

Ainsi :

— $\alpha \approx 0$: convergence lente ;

— $\alpha = 0.5$: convergence intermédiaire ;

— $\alpha = 1$: adaptation très rapide.

Le paramètre α contrôle donc directement la vitesse de convergence du système.

7.2 Pourquoi la polarisation est-elle difficile dans ce modèle ?

Dans le modèle utilisé, chaque agent se rapproche systématiquement de la moyenne des opinions de ses voisins.

La règle :

$$o_i^{t+1} = o_i^t + \alpha \left(\overline{o}_{\mathcal{N}_i}^t - o_i^t \right)$$

réduit progressivement les différences entre les opinions.

Il n'existe pas de mécanisme empêchant un agent d'être influencé par un voisin ayant une opinion très différente.

Par conséquent, dans cette configuration, le système tend naturellement vers un consensus global plutôt que vers une polarisation durable.

7.3 Émergence

L'émergence signifie que le comportement global du système apparaît à partir des interactions locales entre les agents.

Aucun agent ne possède une information globale sur toute la population.

Chaque agent connaît uniquement les opinions de ses voisins et modifie sa propre opinion.

Pourtant, l'ensemble de ces interactions locales produit un comportement global : la convergence progressive des opinions vers un consensus.

Les graphiques permettent donc de visualiser cette émergence.

7.4 Activation des agents dans Mesa 3

Dans Mesa 3, les agents sont activés avec :

```
self.agents.shuffle_do("step")
```

Cette instruction appelle la méthode `step()` de chaque agent dans un ordre aléatoire.

L'utilisation d'un ordre aléatoire évite de toujours favoriser les mêmes agents et permet de représenter plus naturellement les interactions asynchrones entre individus.

7.5 Différence entre model_reporters et agent_reporters

Le `model_reporters` permet de collecter des informations globales sur l'ensemble du modèle.

Dans notre cas, il permet de collecter :

— l'opinion moyenne ;

— la variance des opinions.

Le `agent_reporters` permet quant à lui de collecter des informations propres à chaque agent, notamment son opinion.

L'utilisation des deux est importante car ils permettent d'observer deux niveaux différents :

— le comportement global de la population ;

— le comportement individuel des agents.

7.6 Effet de `torus=True`

Lorsque `torus=True`, la grille est considérée comme ayant des bords communicants.

Un agent situé sur le bord de la grille peut donc avoir comme voisins des agents situés sur le bord opposé.

Cela évite un traitement particulier des agents situés aux extrémités de la grille et permet de conserver un voisinage homogène.

Chapitre 8

Ouverture et amélioration du modèle

8.1 Introduction d'un biais de confirmation

Le modèle actuel considère qu'un agent est influencé par tous ses voisins.

Pour introduire un biais de confirmation, on peut définir un seuil de tolérance ϵ.

Un agent ne serait alors influencé que par les voisins dont l'opinion est suffisamment proche de la sienne :

$$|o_j - o_i| < \epsilon$$

Les voisins dont l'opinion est trop éloignée seraient ignorés.

Cette modification pourrait empêcher le consensus global et favoriser l'apparition de plusieurs groupes d'opinion.

8.2 Vers un véritable réseau social

Le modèle actuel utilise une grille spatiale.

Pour représenter un véritable réseau social, il serait plus adapté d'utiliser une structure basée sur un graphe.

Dans un tel réseau, les agents représenteraient les utilisateurs et les relations entre agents représenteraient les abonnements ou les liens sociaux.

Une telle structure permettrait notamment de représenter des agents très connectés, comme les influenceurs, et des agents possédant seulement quelques connexions.

Conclusion

Ce TP nous a permis de mettre en pratique la modélisation multi-agent avec le framework Mesa 3.

Nous avons construit un modèle dans lequel chaque agent représente un individu possédant une opinion comprise entre 0 et 1. L'opinion de chaque agent évolue en fonction de la moyenne des opinions de ses voisins.

La simulation montre qu'à travers des interactions locales et répétées, un comportement collectif peut apparaître. Avec la configuration utilisée, notamment $\alpha = 0.5$, les opinions tendent progressivement vers un consensus global.

Nous avons également étudié l'utilisation de la nouvelle syntaxe de Mesa 3, la grille `MultiGrid`, le mécanisme d'activation `shuffle_do()` ainsi que le `DataCollector`.

Enfin, l'étude du biais de confirmation et du remplacement de la grille par un réseau social montre comment le modèle pourrait être enrichi afin de reproduire des phénomènes plus complexes tels que la polarisation et la formation de groupes d'opinion.